
Introduction

Sets are as fundamental to computer science as they are to mathematics. Whereas mathematical sets are unchanging, the sets manipulated by algorithms can grow, shrink, or otherwise change over time. We call such sets *dynamic*. The next four chapters present some basic techniques for representing finite dynamic sets and manipulating them on a computer.

Algorithms may require several types of operations to be performed on sets. For example, many algorithms need only the ability to insert elements into, delete elements from, and test membership in a set. We call a dynamic set that supports these operations a *dictionary*. Other algorithms require more complicated operations. For example, min-priority queues, which Chapter 6 introduced in the context of the heap data structure, support the operations of inserting an element into and extracting the smallest element from a set. The best way to implement a dynamic set depends upon the operations that you need to support.

Elements of a dynamic set

In a typical implementation of a dynamic set, each element is represented by an object whose attributes can be examined and manipulated given a pointer to the object. Some kinds of dynamic sets assume that one of the object's attributes is an identifying *key*. If the keys are all different, we can think of the dynamic set as being a set of key values. The object may contain *satellite data*, which are carried

around in other object attributes but are otherwise unused by the set implementation. It may also have attributes that are manipulated by the set operations. These attributes may contain data or pointers to other objects in the set.

Some dynamic sets presuppose that the keys are drawn from a totally ordered set, such as the real numbers, or the set of all words under the usual alphabetic ordering. A total ordering allows us to define the minimum element of the set, for example, or to speak of the next element larger than a given element in a set.

Operations on dynamic sets

Operations on a dynamic set can be grouped into two categories: *queries*, which simply return information about the set, and *modifying operations*, which change the set. Here is a list of typical operations. Any specific application will usually require only a few of these to be implemented.

SEARCH(S, k)

A query that, given a set S and a key value k , returns a pointer x to an element in S such that $x.key = k$, or NIL if no such element belongs to S .

INSERT(S, x)

A modifying operation that adds the element pointed to by x to the set S . We usually assume that any attributes in element x needed by the set implementation have already been initialized.

DELETE(S, x)

A modifying operation that, given a pointer x to an element in the set S , removes x from S . (Note that this operation takes a pointer to an element x , not a key value.)

MINIMUM(S) and MAXIMUM(S)

Queries on a totally ordered set S that return a pointer to the element of S with the smallest (for MINIMUM) or largest (for MAXIMUM) key.

SUCCESSOR(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next larger element in S , or NIL if x is the maximum element.

PREDECESSOR(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next smaller element in S , or NIL if x is the minimum element.

In some situations, we can extend the queries SUCCESSOR and PREDECESSOR so that they apply to sets with nondistinct keys. For a set on n keys, the normal presumption is that a call to MINIMUM followed by $n - 1$ calls to SUCCESSOR enumerates the elements in the set in sorted order.

We usually measure the time taken to execute a set operation in terms of the size of the set. For example, Chapter 13 describes a data structure that can support any of the operations listed above on a set of size n in $O(\lg n)$ time.

Of course, you can always choose to implement a dynamic set with an array. The advantage of doing so is that the algorithms for the dynamic-set operations are simple. The downside, however, is that many of these operations have a worst-case running time of $\Theta(n)$. If the array is not sorted, INSERT and DELETE can take $\Theta(1)$ time, but the remaining operations take $\Theta(n)$ time. If instead the array is maintained in sorted order, then MINIMUM, MAXIMUM, SUCCESSOR, and PREDECESSOR take $\Theta(1)$ time; SEARCH takes $O(\lg n)$ time if implemented with binary search; but INSERT and DELETE take $\Theta(n)$ time in the worst case. The data structures studied in this part improve on the array implementation for many of the dynamic-set operations.

Overview of Part III

Chapters 10–13 describe several data structures that we can use to implement dynamic sets. We'll use many of these data structures later to construct efficient algorithms for a variety of problems. We already saw another important data structure—the heap—in Chapter 6.

Chapter 10 presents the essentials of working with simple data structures such as arrays, matrices, stacks, queues, linked lists, and rooted trees. If you have taken an introductory programming course, then much of this material should be familiar to you.

Chapter 11 introduces hash tables, a widely used data structure supporting the dictionary operations INSERT, DELETE, and SEARCH. In the worst case, hash tables require $\Theta(n)$ time to perform a SEARCH operation, but the expected time for hash-table operations is $O(1)$. We rely on probability to analyze hash-table operations, but you can understand how the operations work even without probability.

Binary search trees, which are covered in Chapter 12, support all the dynamic-set operations listed above. In the worst case, each operation takes $\Theta(n)$ time on a tree with n elements. Binary search trees serve as the basis for many other data structures.

Chapter 13 introduces red-black trees, which are a variant of binary search trees. Unlike ordinary binary search trees, red-black trees are guaranteed to perform well: operations take $O(\lg n)$ time in the worst case. A red-black tree is a balanced search tree. Chapter 18 in Part V presents another kind of balanced search tree, called a B-tree. Although the mechanics of red-black trees are somewhat intricate, you can glean most of their properties from the chapter without studying the mechanics in detail. Nevertheless, you probably will find walking through the code to be instructive.